



UNIVERSIDADE ESTADUAL DE SANTA CRUZ - UESC
DEPARTAMENTO DE ENGENHARIAS E COMPUTAÇÃO - DEC
ENGENHARIA QUÍMICA

Rod. Jorge Amado, Km 16 — Ilhéus/BA

INSTRUMENTAÇÃO COMPUTACIONAL

Aplicações em Engenharia Química

Prof. Dr. E. R. Edwards

Ilhéus, 17 de setembro de 2026

Sumário

1	Medição de Temperatura Computacional	1
---	--------------------------------------	---

Capítulo 1

Medição de Temperatura Computacional

Apresentação e Diretrizes

Esta lista de exercícios práticos visa consolidar os conceitos de transdução de temperatura por meio da **Computação Científica Vetorial**. Todos os cálculos computacionais devem ser realizados utilizando matrizes e vetores com as bibliotecas **numpy** e **pandas**, evitando-se o uso de estruturas de controle de fluxo condicionais (**if**, **else**) ou laços de repetição explícitos.

Exercício 1: Calibração e Incerteza Criogênica (Sensor Diodo de Silício DT-670)

Contexto Industrial: Em sistemas de liquefação de hélio e supercondutividade, diodos de junção p-n de silício alimentados por uma corrente constante de $10\mu\text{A}$ são amplamente empregados devido à alta sensibilidade em baixas temperaturas.

Enunciado: Considere a resposta de um diodo criogênico na faixa de 10 K a 100 K, cuja relação tensão-temperatura em regime de baixa temperatura é aproximada pelo modelo linear $V(T) = V_0 - \alpha T$. Sabendo que para $T_1 = 10\text{ K}$, $V_1 = 1,10\text{ V}$ e para $T_2 = 100\text{ K}$, $V_2 = 0,92\text{ V}$:

1. Determine analiticamente a sensibilidade do sensor $\alpha = -\frac{dV}{dT}$ em mV/K e a tensão constante V_0 .
2. Escreva a equação analítica da temperatura $T(V)$ em função da tensão lida.
3. Via Python, simule um conjunto de 10 leituras de tensão amostradas no tempo com ruído gaussiano adicionado, processe os dados criando uma tabela com **pandas** e plote a temperatura calculada $T(t)$ ao longo do tempo.

Resolução Analítica (Na Mão)

1. Cálculo do coeficiente angular (α):

$$\alpha = -\frac{V_2 - V_1}{T_2 - T_1} = -\frac{0,92 - 1,10}{100 - 10} = -\frac{-0,18}{90} = 0,002 \text{ V/K} = 2,0 \text{ mV/K}$$

- Cálculo do coeficiente linear (V_0):

$$V_1 = V_0 - \alpha T_1 \implies 1,10 = V_0 - 0,002 \times 10 \implies V_0 = 1,12 \text{ V}$$

2. Equação da Temperatura $T(V)$:

$$V = 1,12 - 0,002T \implies T(V) = \frac{1,12 - V}{0,002} = 560 - 500 \cdot V$$

Implementação Computacional

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Parametros analiticos
alpha = 0.002 # V/K
V0 = 1.12 # V

# Sinal temporal de tensao com ruído gaussiano
np.random.seed(42)
tempo = np.linspace(0, 50, 10) # 10 pontos de amostragem
V_nominal = 1.05 - 0.003 * tempo # Queda de tensão linear no tempo
ruído = np.random.normal(0, 0.002, size=len(tempo))
V_medido = V_nominal + ruído

# Vetorizacao do calculo da Temperatura T(V)
T_medido = (V0 - V_medido) / alpha

# Construção do DataFrame com Pandas
df1 = pd.DataFrame({
    'Tempo (s)': tempo,
    'Tensao (V)': V_medido,
    'Temperatura (K)': T_medido
})

print(df1.round(4))

# Visualização Gráfica
```

```
plt.figure(figsize=(7, 4))
plt.plot(df1['Tempo (s)'], df1['Temperatura (K)'], 'o-b', label='Diodo DT-670')
plt.xlabel('Tempo (s)')
plt.ylabel('Temperatura (K)')
plt.title('Monitoramento Criogênico sem Estruturas Condicionais')
plt.grid(True, linestyle=':')
plt.legend()
plt.tight_layout()
plt.show()
```

Exercício 2: Não Linearidade e Comparação Termométrica (PT100 vs NTC)

Contexto Industrial: Em processos químicos industriais na faixa termométrica (-50°C a 150°C), a escolha entre termorresistências de platina (PT100) e termistores NTC depende do compromisso entre linearidade e sensibilidade.

Enunciado: Um sensor PT100 segue a equação de Callendar-Van Dusen simplificada: $R_{PT}(T) = R_0(1 + AT)$, com $R_0 = 100\ \Omega$ e $A = 3,908 \times 10^{-3}\ ^{\circ}\text{C}^{-1}$. Um termistor NTC obedece ao modelo exponencial: $R_{NTC}(T) = R_0 \exp\left[\beta\left(\frac{1}{T+273,15} - \frac{1}{T_0+273,15}\right)\right]$, com $R_0 = 10000\ \Omega$, $T_0 = 25^{\circ}\text{C}$ e $\beta = 3950\ \text{K}$.

1. Calcule analiticamente a resistência de ambos os sensores para a temperatura de $T = 50^{\circ}\text{C}$.
2. Determine a sensibilidade absoluta $S(T) = \frac{dR}{dT}$ de cada sensor a 50°C .
3. Implemente um script em Python para gerar uma curva comparativa das resistências normalizadas $R(T)/R(0^{\circ}\text{C})$ em uma varredura contínua de temperatura de 0°C a 100°C .

Resolução Analítica (Na Mão)

1. Para $T = 50^{\circ}\text{C}$:

$$R_{PT}(50) = 100 \cdot (1 + 3,908 \times 10^{-3} \times 50) = 100 \cdot (1 + 0,1954) = 119,54\ \Omega$$

Para o NTC ($T = 50^{\circ}\text{C} = 323,15\ \text{K}$ e $T_0 = 298,15\ \text{K}$):

$$\frac{1}{323,15} - \frac{1}{298,15} = 0,0030944 - 0,0033540 = -0,0002596\ \text{K}^{-1}$$

$$R_{NTC}(50) = 10000 \cdot \exp(3950 \times (-0,0002596)) = 10000 \cdot \exp(-1,0254) \approx 3586,52\ \Omega$$

2. Sensibilidades em $T = 50^\circ\text{C}$:

$$S_{PT} = \frac{dR_{PT}}{dT} = R_0 A = 100 \times 3,908 \times 10^{-3} = 0,3908 \Omega/^\circ\text{C}$$

$$S_{NTC}(T) = \frac{dR_{NTC}}{dT} = -\frac{\beta}{(T + 273,15)^2} R_{NTC}(T)$$

$$S_{NTC}(50) = -\frac{3950}{(323,15)^2} \times 3586,52 = -0,03782 \times 3586,52 \approx -135,64 \Omega/^\circ\text{C}$$

Implementação Computacional

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Parâmetros dos sensores
R0_pt, A_pt = 100.0, 3.908e-3
R0_ntc, T0_ntc, beta_ntc = 10000.0, 25.0, 3950.0

# Vetorização da temperatura
T = np.linspace(0, 100, 100)

# Modelos matemáticos vetoriais
R_pt = R0_pt * (1 + A_pt * T)
R_ntc = R0_ntc * np.exp(beta_ntc * (1/(T + 273.15) - 1/(T0_ntc + 273.15)))

# Normalização pelas respectivas resistências a 0 °C
R_pt_norm = R_pt / R_pt[0]
R_ntc_norm = R_ntc / R_ntc[0]

# Tabela comparativa usando Pandas
df2 = pd.DataFrame({
    'Temperatura (C)': T[:20],
    'R_PT100 (Ohm)': R_pt[:20],
    'R_NTC (Ohm)': R_ntc[:20]
})

print(df2.to_string(index=False))

# Gráfico da resposta comparativa
plt.figure(figsize=(7, 4))
plt.plot(T, R_pt_norm, 'r-', label='PT100 (Linear)', linewidth=2)
plt.plot(T, R_ntc_norm, 'b--', label='NTC (Exponencial)', linewidth=2)
plt.xlabel('Temperatura (°C)')
plt.ylabel('Resistência Normalizada R(T)/R(0°C)')
```

```
plt.title('Comparação Termométrica: PT100 vs Termistor NTC')
plt.grid(True, linestyle=':')
plt.legend()
plt.tight_layout()
plt.show()
```

Exercício 3: Linearização da Função do Termopar Tipo K

Contexto Industrial: Termopares são amplamente aplicados no monitoramento de processos de alta temperatura devido à sua faixa operacional estendida.

Enunciado: A força eletromotriz $E(T)$ gerada por um termopar Tipo K com junção de referência a 0°C pode ser aproximada linearmente por $E(T) = S_0 \cdot T$, em que $S_0 = 41,2 \mu\text{V}/^\circ\text{C}$.

1. Determine a força eletromotriz teórica gerada em $T = 400^\circ\text{C}$ em mV.
2. Sabendo que o modelo polinomial do NIST de segunda ordem apresenta correção dada por $E(T) = a_0 + a_1T + a_2T^2$, com $a_0 = 0 \text{ mV}$, $a_1 = 0,0395 \text{ mV}/^\circ\text{C}$ e $a_2 = 0,000004 \text{ mV}/^\circ\text{C}^2$, calcule o erro absoluto da aproximação linear simples a 400°C .
3. Crie um script Python vetorial para calcular a fem para um vetor de temperaturas e determinar o erro relativo percentual sem utilizar laços condicionais.

Resolução Analítica (na Mão)

1. Cálculo linear simples para $T = 400^\circ\text{C}$:

$$E_{\text{linear}}(400) = 41,2 \mu\text{V}/^\circ\text{C} \times 400^\circ\text{C} = 16480 \mu\text{V} = 16,48 \text{ mV}$$

2. Modelo polinomial do NIST a 400°C :

$$E_{\text{NIST}}(400) = 0,0395 \times 400 + 0,000004 \times (400)^2 = 15,8 + 0,64 = 16,44 \text{ mV}$$

Erro absoluto:

$$\text{Erro} = |E_{\text{linear}} - E_{\text{NIST}}| = |16,48 - 16,44| = 0,04 \text{ mV}$$

Implementação Computacional

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Temperatura vetorial de 0 a 1000 °C
```

```

T = np.linspace(0, 1000, 50)

# Modelos computacionais vetoriais
E_linear = 0.0412 * T # em mV
E_nist = 0.0395 * T + 0.000004 * (T**2) # em mV

# Cálculo do Erro Relativo Percentual de forma vetorial
erro_percentual = np.abs((E_linear - E_nist) / np.maximum(E_nist, 1e-9)) * 100

# DataFrame para visualização sintética
df3 = pd.DataFrame({
    'Temperatura (°C)': T[:10],
    'E_Linear (mV)': E_linear[:10],
    'E_NIST (mV)': E_nist[:10],
    'Erro (%)': erro_percentual[:10]
})

print(df3.round(3).to_string(index=False))

# Gráfico de Discrepância
plt.figure(figsize=(7, 4))
plt.plot(T, E_nist, 'k-', label='Modelo NIST')
plt.plot(T, E_linear, 'r--', label='Aproximação Linear Simples')
plt.xlabel('Temperatura (°C)')
plt.ylabel('Força Eletromotriz (mV)')
plt.title('Linearização da Resposta de Termopar Tipo K')
plt.grid(True, linestyle=':')
plt.legend()
plt.tight_layout()
plt.show()

```

Exercício 4: Pirometria Óptica e Correção de Emissividade (Lei de Stefan-Boltzmann)

Contexto Industrial: Em siderúrgicas e fornos de alta temperatura (faixa pirométrica), a medição sem contato depende diretamente da emissividade da superfície do material estudado.

Enunciado: Um pirômetro óptico é calibrado para medir a radiância de um corpo negro ($\epsilon = 1,0$) segundo a Lei de Stefan-Boltzmann $W = \sigma T^4$, em que $\sigma = 5,670374 \times 10^{-8} \text{ W/m}^2\text{K}^4$. Ao apontar o instrumento para uma placa de aço oxidado cuja emissividade real é $\epsilon = 0,85$, o instrumento indica uma temperatura aparente (temperatura de brilho) $T_{ap} = 1200 \text{ K}$.

1. Deduza analiticamente a relação entre a Temperatura Real (T_{real}) e a Temperatura

Aparente (T_{ap}).

2. Calcule o valor exato da Temperatura Real T_{real} da placa em Kelvin.
3. Desenvolva um script em Python que compute vetorialmente a temperatura real corrigida para diferentes coeficientes de emissividade ($\epsilon = 0,6; 0,7; 0,85; 1,0$).

Resolução Analítica (na Mão)

1. Igualando as densidades de potência radiada captadas pelo pirômetro:

$$W_{\text{medido}} = \epsilon \cdot \sigma T_{\text{real}}^4 = 1,0 \cdot \sigma T_{\text{ap}}^4$$

Simplificando a constante de Stefan-Boltzmann (σ):

$$\epsilon T_{\text{real}}^4 = T_{\text{ap}}^4 \implies T_{\text{real}} = \frac{T_{\text{ap}}}{\epsilon^{1/4}}$$

2. Cálculo para $T_{ap} = 1200$ K e $\epsilon = 0,85$:

$$T_{\text{real}} = \frac{1200}{(0,85)^{0,25}} = \frac{1200}{0,9598} \approx 1250,26 \text{ K}$$

Implementação Computacional

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Temperatura aparente constante lida pelo pirômetro
T_aparente = 1200.0 # Kelvin

# Vetor de emissividades de diferentes superfícies industriais
emissividades = np.array([0.5, 0.65, 0.80, 0.85, 0.95, 1.0])

# Cálculo totalmente vetorial sem uso de laços condicionais
T_real = T_aparente / (emissividades ** 0.25)
erro_leitura = T_real - T_aparente

# Estruturação e manipulação com Pandas
df4 = pd.DataFrame({
    'Emissividade (e)': emissividades,
    'T_Aparente (K)': T_aparente,
    'T_Real Corrigida (K)': T_real,
    'Erro de Leitura (K)': erro_leitura
})
```

```

print(df4.round(2).to_string(index=False))

# Plotagem da influência da emissividade
plt.figure(figsize=(7, 4))
plt.plot(df4['Emissividade (e)'], df4['T_Real Corrigida (K)'], 's-g', linewidth=2)
plt.axhline(1200, color='r', linestyle='--', label='Temperatura Aparente (1200
↪ K)')
plt.xlabel('Emissividade da Superfície ( $\epsilon$ )')
plt.ylabel('Temperatura Real (K)')
plt.title('Correção Pirométrica de Temperatura por Emissividade')
plt.grid(True, linestyle=':')
plt.legend()
plt.tight_layout()
plt.show()

```

Exercício 5: Dinâmica e Constante de Tempo em Processos Térmicos

Contexto Industrial: A resposta dinâmica de um sensor térmico imerso em um fluido é modelada como um sistema de primeira ordem, caracterizado por sua constante de tempo térmica τ .

Enunciado: Um poço de proteção contendo um PT100 é submetido a um degrau de temperatura de $T_i = 25^\circ\text{C}$ para $T_f = 125^\circ\text{C}$ em $t = 0\text{ s}$. A resposta dinâmica da temperatura indicada é governada pela equação diferencial de primeira ordem $T(t) = T_f + (T_i - T_f)e^{-t/\tau}$, onde $\tau = 5,0\text{ s}$.

1. Calcule analiticamente a temperatura do sensor nos instantes $t = \tau$ e $t = 3\tau$.
2. Determine o tempo necessário para que o sensor atinja 99% da variação total de temperatura ($t_{99\%}$).
3. Faça um código computacional para simular a resposta dinâmica e calcular a taxa de variação Instantânea $\frac{dT}{dt}$ usando a função diferencial do NumPy (`np.gradient`).

Resolução Analítica (na Mão)

1. Para $t = \tau = 5\text{ s}$:

$$T(\tau) = 125 + (25 - 125)e^{-1} = 125 - 100 \times 0,36787 = 88,21^\circ\text{C}$$

Para $t = 3\tau = 15\text{ s}$:

$$T(3\tau) = 125 + (25 - 125)e^{-3} = 125 - 100 \times 0,049787 = 120,02^\circ\text{C}$$

2. Tempo para 99% da variação (sendo $\Delta T = 100^\circ\text{C}$, 99% equivale a 99°C , ou seja, $T = 124^\circ\text{C}$):

$$124 = 125 - 100e^{-t/\tau} \implies 0,01 = e^{-t/5} \implies -\frac{t}{5} = \ln(0,01) \approx -4,605$$

$$t_{99\%} = 5 \times 4,605 \approx 23,03 \text{ s}$$

Implementação Computacional

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Parâmetros do sistema
Ti, Tf, tau = 25.0, 125.0, 5.0
tempo = np.linspace(0, 30, 100)

# Resposta ao degrau calculada vetorialmente
T_resposta = Tf + (Ti - Tf) * np.exp(-tempo / tau)

# Derivada numérica da temperatura no tempo dT/dt (Derivada Vetorial)
dT_dt = np.gradient(T_resposta, tempo)

# Organização e apresentação dos resultados em DataFrame
df5 = pd.DataFrame({
    'Tempo (s)': tempo[::15],
    'Temperatura indicada (C)': T_resposta[::15],
    'Taxa dT/dt (C/s)': dT_dt[::15]
})

print(df5.round(2).to_string(index=False))

# Representação gráfica com múltiplos subplots
fig, ax1 = plt.subplots(figsize=(7, 4))

ax1.set_xlabel('Tempo (s)')
ax1.set_ylabel('Temperatura (°C)', color='tab:blue')
ax1.plot(tempo, T_resposta, color='tab:blue', linewidth=2, label='Temperatura')
ax1.tick_params(axis='y', labelcolor='tab:blue')
ax1.grid(True, linestyle=':')

ax2 = ax1.twinx()
ax2.set_ylabel('dT/dt (°C/s)', color='tab:red')
ax2.plot(tempo, dT_dt, color='tab:red', linestyle='--', label='Taxa de Variação')
ax2.tick_params(axis='y', labelcolor='tab:red')
```

```
plt.title('Resposta Dinâmica do Poço Térmico (Sistema de 1ª Ordem)')  
fig.tight_layout()  
plt.show()
```